

00000 0 000000 00

00 000: feed-mina/MIMO
00 000: copilot/screen-code-handover
00: 00000 00 00 000
00: 00000, 00 000 00 API/000 000, 0000 00
000 00: 00 00000 000 000 00000, 000 0 000·DB 0000 0000 00000.

00 00

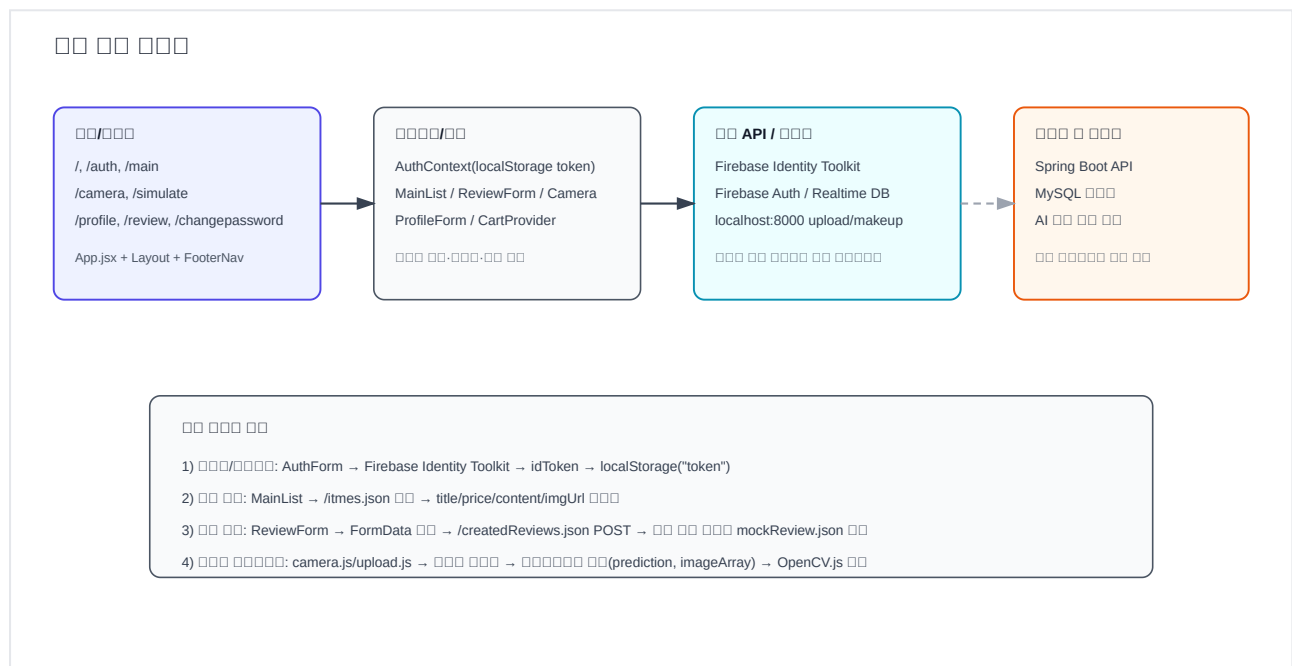
00	00
00	React 00 00 0000 000 00 0000000 000
00 00	00 00 000 00 000 00, 00 0 000 00 00 ·API·000 000 0000 00 000 0000.
00 00	1) 00 00 → 2) 000 0000 → 3) API → 4) DB 0000 ERD → 5) 0000 00
00 00	mimo-frontend/src 0 00 0000 0000, 0 000 000 000/00, 00 00 00
000 00	Spring Boot 00 00, MySQL 000000/00, localhost:8000 AI 00 00, 00 000 00

00 00

- 00 00
- 000 0000
- 0 /
- 00 /auth
- 00 00 00 /main
- 000 00000 /camera
- 00000 00/00 /simulate
- 000 /profile

- `POST /change-password`
- `POST /review`
- API
- DB `ERD`
- `POST /login`

MIMO는 React, Firebase, AI, Spring Boot, MySQL, Node.js 등 다양한 기술을 활용하여 다양한 서비스를 개발하고 있습니다. README는 Spring Boot, MySQL, Node.js AI 등 다양한 기술을 활용하여 다양한 서비스를 개발하고 있습니다. README는 Spring Boot, MySQL, Node.js AI 등 다양한 기술을 활용하여 다양한 서비스를 개발하고 있습니다.



--	--	--	--	--

구분	구분	구분
구분	/home/runner/work/MIMO/MIMO/mimo-frontend/src/App.jsx	구분
구분	components/Layout/*	구분
구분	store/auth-context.jsx	구분

00	00 00	00
		localStorage 0 token 0 0 0000 000 000 00000.
000 00/00	components/Main/*, components/Review/*, components/Cart/*, data/api.jsx	Firebase Realtime Database0 JSON 000 00 00000.
AI 00000	components/Modeling/ca mera.js , components/Modeling/up load.js	00/0000 0000 localhost:8000/api/v1/ upload , localhost:8000/api/v1/ makeup 00 000, 000 OpenCV.js0 00000.
000 0 000	README00 00	Spring Boot, MySQL, AI 00 0 00 00 00000000 000 0 0 000.

00 0000 00

- 000/0000: AuthForm 0 Firebase Identity Toolkit REST API0 0000, 000 idToken 0
AuthContext.login() 0 0000.
- 00 00: MainList 0 /itmes.json 0 00 00 00 00 00000000.
- 00 00: ReviewForm 0 FormData 0 000 /createdReviews.json 0 POST000, 00 00
0 00 000 mockReview.json 0 00 000 00 0000 0000.
- 000 00: camera.js 0 upload.js 0 000 API0 path , 0000 API0 prediction ,
imageArray 0 00 00000000 00 0000.

000000 000 0000 0000

- FooterNav 0 MainNavigation 00 /cart 000 000 App.jsx 00 /cart 0000 000
0. 00 00 0 000000 00 000 /main 00 000000.
- data/APIUtils.js 0 ../constants 0 import000 00 000 00, 00 0 0000000 0 000
0000 0000.
- camera.js 0 000 0000 00 this.setState 0 000 00 00 0000 00 00 000 0000
0.
- mimo-modeling 0000 00 000 000 0000.

0000 000000

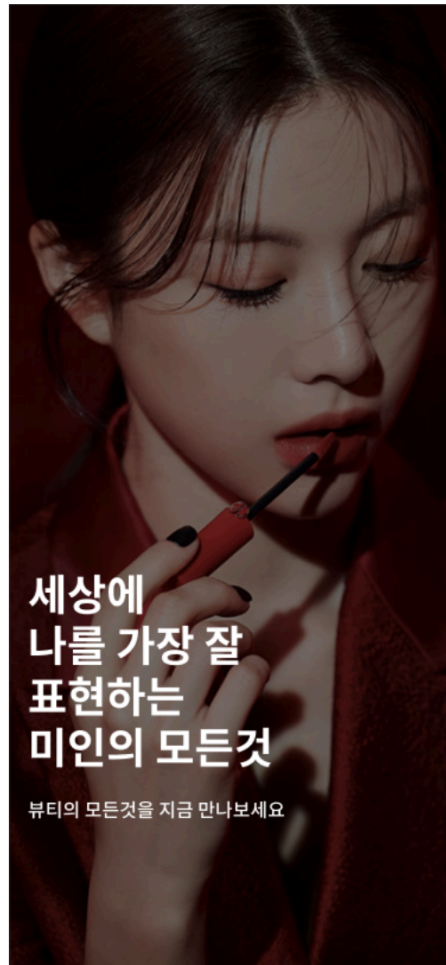
□□ **1.** □ /

□ □□□ □□□ □□ □□□□ □□ □□ □□□ □□ □□□□□□. `props.authenticated` □ □□□
`useEffect` □□ `/main` □□ □□□□□ □□□□ □□□, □□ `App.jsx` □□ □□ `props` □□□□ □□□□.

MIMO

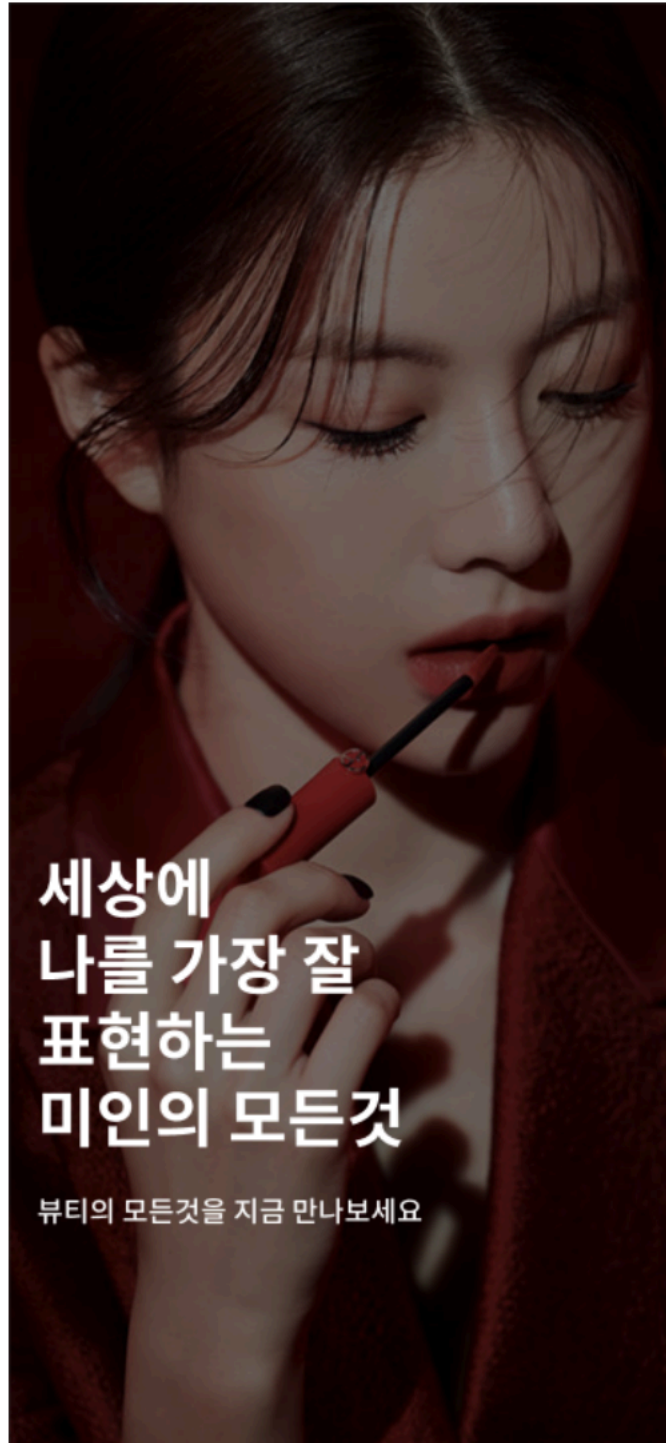


Login



MIMO

0000 0000 00, 000000 00000 00 0 0000 00 0000
0000 000000.



세상에
나를 가장 잘
표현하는
미인의 모든것

뷰티의 모든것을 지금 만나보세요

MIMO

00 000 00

00	00	0000 00 0
<code>components/Home/Home.js</code> <code>sx</code>	00 000 000	000 000 000, 00, 00 00 000 00000.
<code>App.jsx</code>	00 000	<code>/</code> 0000 <code>Home</code> 0 000000.
<code>components/Layout/Layout.js</code> <code>ut.jsx</code>	00 00	0 000 000 0.00 00000 0000 00 00000.

00.00000 00

00	00	00	00/00 00
<code>Home(props)</code>	<code>props.authentic</code> <code>ated (000, 000)</code>	00 000 0 00000 0000 <code>navigate('/main</code> <code>')</code> 0 00000.	JSX 0000, 000 0 00 000 00 00 00 0 000.

00 00

- 00 00: 00 00 000 0000.
- 000 00: 00 00, 00 000000 00 00 0000 00000.
- 00 00 00: 00 000 0000 00, 00 00 00 000000.

00 API / DB

0 000 API 00 0000 0000.

00 2. 00 /auth

00 000 000/0000 0000 00000 00 000 0000, 0000 **Google** 00 000 000 000000. 000
00 0 `idToken` 0 0000 `/main` 00 00000.

MIMO



Login

Login

Email

Password

Login

[Create new account](#)



구글로 시작하기



Email

Password

Login

Create new account



구글로 시작하기



00 000 00

00	00	0000 00 0
pages/AuthPage.jsx	000 00	00 00000 AuthForm 0 0 00000.
components/Auth/AuthFo rm.jsx	00 000 00 00	000/0000 URL 00, Firebase REST 00, Google 0 0 0000 00000.
store/auth- context.jsx	00 00 00	000 localStorage 0 000 0 isLoggedIn 0 00000.
fbase.js	Firebase 0 000	Firebase Auth 00000 000 Google 00 0000 00000.

00•00000 00

00	00	00	000 / 00 00
switchAuthModeH andler()	00	isLogin 000 000 00.	00 000 00 URL 0 000.

00	00	00	000 / 00 00
<code>submitHandler(e vent)</code>	<code>00 email , password</code>	<code>isLogin 0 000 accounts:signIn WithPassword , 00 00 accounts:signUp 0 00000.</code>	<code>00 0 authCtx.login(d ata.idToken) , navigate('/main<br ');="" 0="" 00="" 00<br="" alert=""/>00.</code>
<code>onSocialClick()</code>	<code>00</code>	<code>GoogleAuthProvi der 0 00 0000 00 000.</code>	<code>00 0 data.user 0 00 state 0 00000 0 0 00 00 000 000 0.</code>
<code>AuthContext.log in(token)</code>	<code>token 000</code>	<code>state 0 localStorage['t oken'] 0 00 0000 0.</code>	<code>00 00 000 00 00 000 000.</code>

00 0000

- 0000 0000 00000 00000.
- 00 00: `isLogin 0 true` 0 000 API, `false` 0 0000 API 0 00000.
- 000 0000 JSON 00 `idToken` 0 000 0 `/main` 00 00000.
- 000 000 `Authentication failed!` 000 000 alert 0 00000.
- Google 000 00 00 0000, Firebase JS SDK 0 00 000 00000.

00 00

- 00 00 **Email/Password** 000 → `emailInputRef , passwordInputRef`
- 000/00 00 → `submitHandler()` → Firebase Identity Toolkit
- Google** 00 → `signInWithPopup(authService, provider)`
- 00 00 → `auth-context.jsx` 0 `token` , `isLoggedInIn` 00 → 00 000 00

00 **API**

000	00	00 00	000 00 0000 00 00	00 00
POST	<code>https://iden titytoolkit. googleapis.c om/v1/accoun</code>	<code>email , password , returnSecure Token</code>	<code>idToken</code>	<code>000 00</code>

000	00	00 00	000 00 0000 00 00	00 00
	ts:signInWithPassword?key=...			
POST	https://identitytoolkit.googleapis.com/v1/accounts:signUp?key=...	email, password, returnSecureToken	idToken	0000 00
SDK popup	Firebase Auth Google provider	00	data.user	Google 00 000

000 00 000

000	0/00	00/00	00
localStorage	token	00/00/00	00 00 000
Firebase Auth currentUser	displayName, photoURL 0	00	000 0000 00 00

00 3. 00 00 00 /main

00 000 000 000 00 00 000 000000. 0000 Firebase Realtime Database /itmes.json 0 0000, 00/00/00 000 000000.

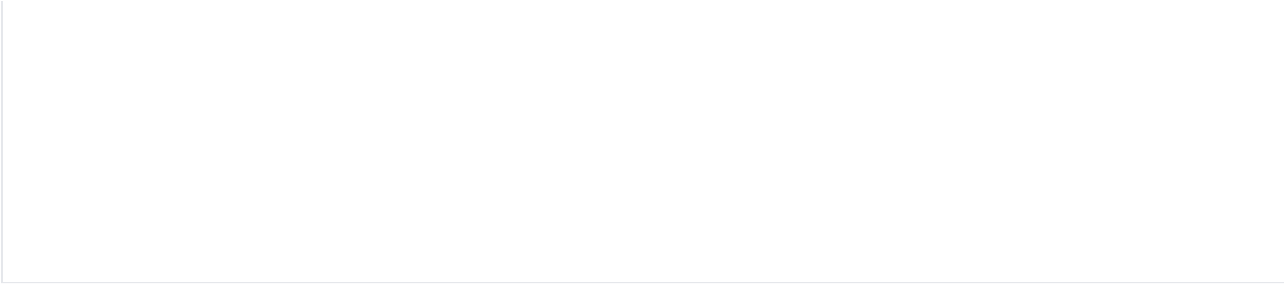
MIMO



Login

Failed to fetch







00 0000 00

00	00	0000 00 0
<code>pages/Mainpages.jsx</code>	0000 00	0000 000 000 00 00 <code>MainList</code> 0 000000.
<code>components/Main/MainList.jsx</code>	00 00 0000 000	<code>/itmes.json</code> 00, JSON 0000 00, 00 00 UI 0000.
<code>components/Layout/nav/FooterNav.jsx</code>	00 0000 00	00 0/000/00/0000 000000 000000.

00•00000 00

00	00	00	000 / 00 00
<code>MainList()</code>	<code>props</code> 000000 000	0000 0 <code>fetchItems()</code> 0 0 0000.	JSX 0 00000 00 <code>state(items , isLoading , httpError)</code> 0 0000 00.
<code>fetchItems()</code>	00	<code>/itmes.json</code> 000 0000 <code>{id,title,price ,content,imgUrl }</code> 0000 000000.	00 0 0000 0000, 00 0 00 0000 0000 00 0000.
<code>MainListItem({ item })</code>	<code>item.Url , item.imgUrl , item.title , item.price , item.content</code>	00 00 0000 0000.	<code><a></code> 0000 00 00 0 00 000000.

00 0000

- 00 00 0 `isLoading=true` 0 000000.
- Firebase 0000 00000.
- 00 00: `response.ok` 0 00000 0000 0000 `httpError` 0 0000000.
- 00 00: 0 0000 00 0000 0000 0000 `items` 0 000000.
- 0000 0000 0000 0000 00 000000 000000.

□□ □□

- □□ □□□/□□/□□/□□ → `responseData[key].imgUrl/title/price/content`
- □□ □□ □□ → `item.Url` □ □□□□□ □□ □□□□ □□ □ `Url` □□□□ □□□□ □□ □□ □□□□ □□ □□ □□□□ □□□□.
- □□ □□ → □□ □□□□ □□□□ □□□□□□□□.

□□ API / DB

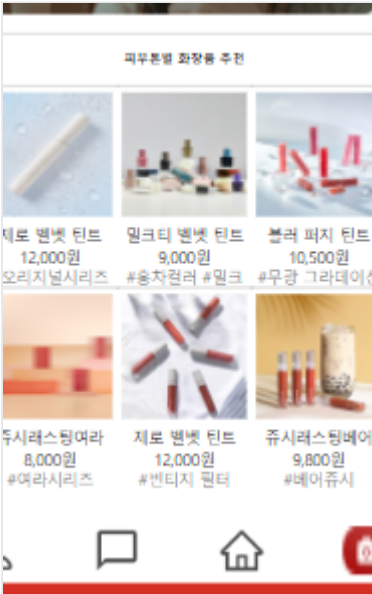
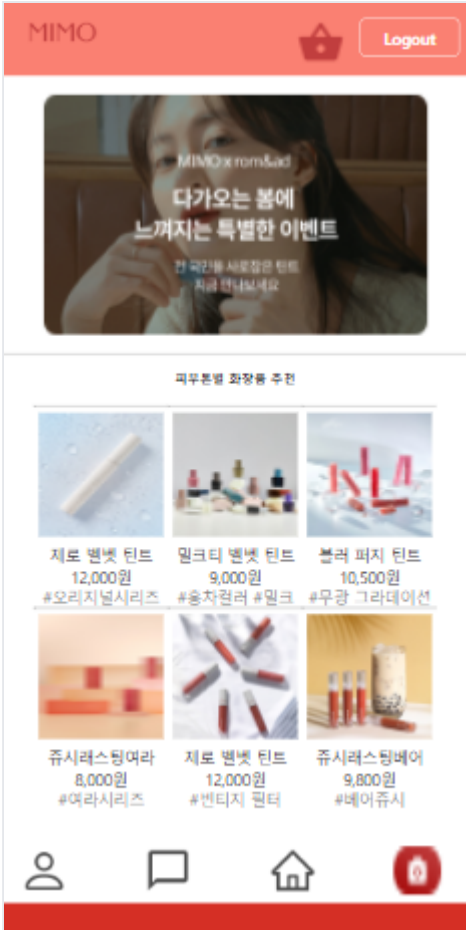
□□□	□□	□□	□□□□ □□□□ □□ □□	□□ □□
GET	<code>https://mimo-49f6d-default-rtdb.firebaseio.com/items.json</code>	□□	<code>{[id]: {title, price, content, imgUrl}}</code>	□□ □□ □□

□□ □□□□ □□

□□	□□	□□
<code>/items/{itemId}</code>	<code>title, price, content, imgUrl</code>	□□ □□ □□ □□□□ □□□□□□.

□□ 4. □□□□ □□□□□□ /camera

□□□□ □□□□ □□□□ □□ □□ □□□□ □□□□ □□ □, □□□□/□□□□□□ API□ □□ □□□□ □ □□ □□ □□□□ □□ □ □□□□□. □□ □□□□ □□ □□□□□□ □□ □□□□ □□ □□□□ □□ □□□□.



00 000 00

00	00	0000 00 0
<code>components/Modeling/camera.js</code>	00 000 00000	0000 000 00, 000 00, 0 00/0000 API 00, 00/00 0 0 UI 00000.

00	00	0000 00 0
<code>components/Modeling/upload.js</code>	00 0000 00000	000 00 00, 0000, 000 0 OpenCV.js 00 000 class component 0 00000.
<code>components/Modeling/constants.jsx</code>	00 00	CAMERAIMAGE , PROD 0 00 000 00 00 00 00000.

00•0000 00

00	00	00	000 / 00 00
<code>getVideo()</code>	00	<code>navigator.mediaDevices.getUserMedia({video: {width:1080,height:1080}})</code> 0 000 00.	00 0 <video> 0 stream 0 0000, 00 0 0000 000 000 0.
<code>takePhoto()</code>	00 000 000	0000 0000 00 data URL 0 000 FormData 0 file 0 000 000 API 0 0 0000.	hasPhoto=true , 0 00 00, 00 0000 0 0 00 000 00000.
<code>ColorChange(event)</code>	<code>event.target.value (#RRGGBB)</code>	00 00 state 0 0000 0.	00 Apply 0000 00 000.
<code>featureChange(event)</code>	<code>lip</code> 00 <code>hair</code>	00 000 state 0 000 00.	00 Apply 00 00 00 0 00000.
<code>Upload.onClickHandler()</code>	000 00	0000 API → 0000 API 00 00000.	path , prediction , image 0 class state 0 0000.
<code>Options.onApplyHandler()</code>	<code>prediction</code> , <code>image</code> , <code>feature</code> , <code>color</code>	00 id 00 0000 0 00 OpenCV.js 00 0000 00000.	canvasOutput 0 0 00 000000.

```

graph TD
    A[camera.js] --> B[upload.js  
canvas  
FileReader]
    B --> C[FormData  
file  
field name: file]
    C --> D[POST /api/v1/upload  
path]
    D --> E[POST /api/v1/makeup  
{ path }]
    E --> F[prediction + imageArray  
prediction: id]
    F --> G[lip/hair  
hex color]
    F --> H[lip: partld 11,12 / hair: 13]
    G --> I[OpenCV.js  
canvasOutput]
    H --> I
  
```

• camera.js: `useState` this image data state image data state image data state image data state.
 • upload.js: `upload` path state this.state.path image data state image data state image data state.
 • localhost:8000 image data state image data state image data state image data state.
 • Apply prediction/imageArray image data state image data state.

UI Text	Method	Parameters	Notes
Take a Picture Button	takePhoto()	None	Called when the button is clicked. Returns a <code>Uri</code> of the image.
Select feature Dropdown	featureChange() / Options.handleFeatureChange()	lip, hair	Called when a feature is selected. Returns the selected feature's ID.
Color input	ColorChange() / Options.handleColorChange()	hex color	Called when the color is changed. Returns the new color's RGB values.
Apply Button	Options.onApplyHandler()	None	Called when the Apply button is clicked. Returns the canvas output.

API

Method	URL	Request Body	Request Headers	Response
POST	http://localhost:8000/api/v1/upload	FormData(file)	path	image URL
POST	http://localhost:8000/api/v1/makeup	{ path }	prediction, imageArray	prediction, imageArray

API 100 100 100

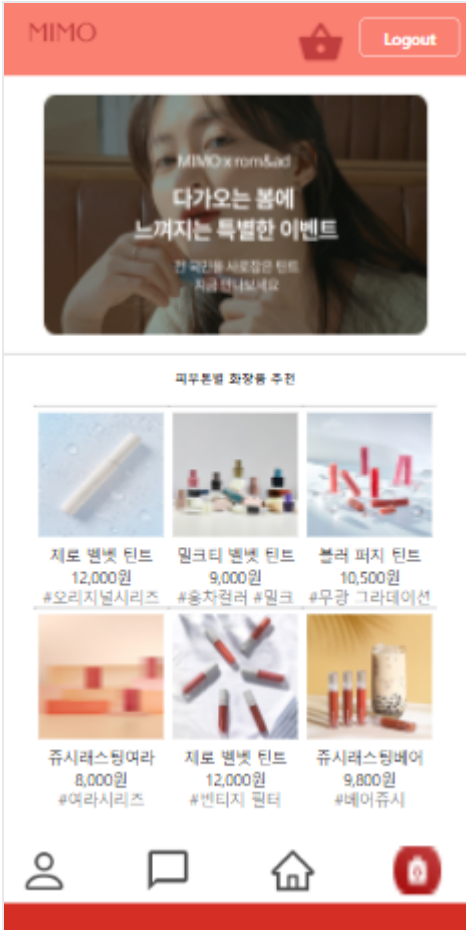
API 100 100 100 state API 100 100 100, Firebase DB API 100 100 100.

API 100 100 100

- camera.js API 100 100 100 image_file state API 100 100 100 FormData API 100 100 100.
- API 100 100 100 this.setState(...) API 100 100 100 100 100 100 100 100 100.
- upload.js API 100 100 100 100 100 100 100 100, localhost:8000 API 100 100 100 100 100 100 100 100.

5. API 100 100 /simulate

/simulate API 100 100 100 100 100 100, API 100 100 100 100 100 100 100 100 100 100 100. API 100 100 100 100 100 100 100 100 100 100 100.



00 000 00

00	00	0000 00 0
<code>components/Modeling/Si mulate.jsx</code>	00/00 00 00	Firestore <code>currentUser</code> 0 provider 00 00 0000 0 00/00 00 00000.
<code>components/Modeling/co nstants.jsx</code>	00 00	<code>CAMERAIMAGE</code> , <code>PROD</code> 000 /cameraimage , /prod:id 00 000 0000 0.

00•0000 00

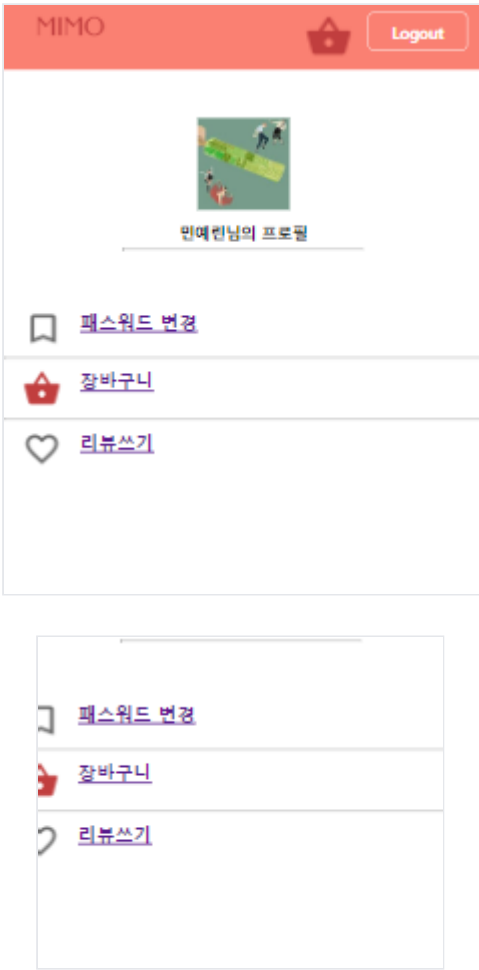
00	00	00	000 / 00 00
<code>Simulate()</code>	Firestore <code>user</code>	<code>user !== null</code> 0 0 000 00 00 <code>providerData</code> 00 0 0 000 0000.	00 00 0000 0000 000 JSX 00000.

API / DB

이 단락을 통해 Firebase Auth를 사용하여 로그인 상태를 관리할 수 있습니다.

6. 사용자 프로필 (/profile)

이 단락을 통해 사용자 프로필 페이지를 구현할 수 있습니다. 이 페이지는 Firebase Auth의 `currentUser`를 사용하여 로그인 상태를 관리합니다.



이 단락을 통해

파일명	기능	데이터베이스
components/Profile/Use rProfile.jsx	사용자 프로필	photoURL, displayName 이름과 사진 URL을 반환합니다.
components/Profile/Pro fileContent.jsx	프로필 내용	/cart, /review 데이터 를 반환합니다.

00	00	0000 00 0
App.jsx	00 00	000 000 00 /profile 0 000 00000.

00•00000 00

00	00	00	000 / 00 00
UserProfile()	authCtx.isLogge dIn , authService.cur rentUser	000 000 000 000 0 000 00000.	JSX0 00000.
ProfileContent()	00	0000/00 00 000 00000.	00 000 00 000 0 000 000 000000.

00 00

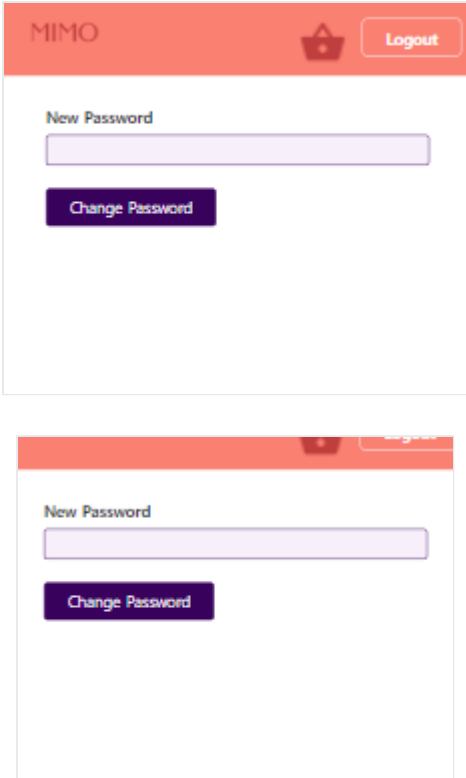
- 000 00 00 → authService.currentUser.photoURL
- 00 00 → authService.currentUser.displayName
- 0000 00 00 → /changepassword
- 0000 00 → /review
- 0000 00 → /cart (00 App 000 000)

00 API / DB

0000 000 00 Firebase Auth 000 000 0000.

00 7. 00000 00 /changepassword

0000 00 000 0 0000 0 00 0000 Firebase Identity Toolkit0 00 0000 API0 00000. 00/00
00 000 00, 00 00 0 000 0000 00 00000.



00 0000 00

00	00	0000 00 0
components/Profile/ProfileForm.jsx	0000 00 0	0 00000 00 00 0000 REST API 00000.
store/auth-context.jsx	00 00	00 000 00 00 idToken 0 0 00000.

00•00000 00

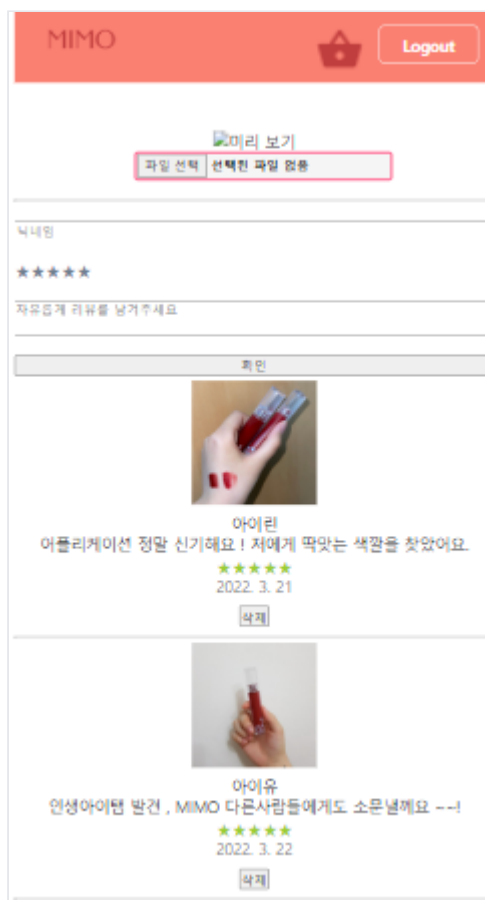
00	00	00	000 / 00 00
submitHandler(event)	new-password 00 0, authCtx.token	accounts:update API idToken , password , returnSecureToken:false 0 0000 0.	00 00 00 00 navigate('/', {replace:true}) 0 00000.

API

요청 방법	요청 URL	요청 데이터	응답 데이터	비고
POST	<code>https://identitytoolkit.googleapis.com/v1/accounts:update?key=...</code>	<code>idToken,</code> <code>password,</code> <code>returnSecureToken:false</code>	응답	로그인 성공 시

8. 리뷰 /review

이 앱은 Firebase를 사용하여 리뷰 데이터를 저장합니다. Firebase를 사용하여 리뷰 데이터를 저장하는 방법은 Firebase 콘솔에서 `mockReview.json` 파일을 업로드하여 리뷰 데이터를 저장할 수 있습니다.





00 000 00

00	00	0000 00 0
components/Review/Review.jsx	00 00 00	ReviewForm 0 ReviewList 0 0000 00 0 000 0000.
components/Review/ReviewForm.jsx	00 0	00, 000, 00, 00 000 state 0 00 FormData 0 00000.
components/Review/FileInput.jsx	000 000 0000	00 000 000 000 URL.createObjectURL() 0 00000 0000.
components/Review/RatingInput.jsx / Rating.jsx	00 00	hover/select 000 0 00 UI0 00000.
components/Review/ReviewList.jsx	00 00	mockReview.json 000 0 00 000 000000.
data/api.jsx	00 API helper	/createdReviews.json POST 0 /review.json GET helper 0 00000.

00•0000 00

00	00	00	000 / 00 00
handleChange(name, value)	000, 0	0 state 0 00 00 00 000.	values state 0 000 00.

00	00	00	000 / 00 00
handleSubmit(e)	values.content , values.imgUrl , values.nickname , values.rating	FormData 0 000 createReviews(f ormData) 0 0000 0.	00 0 0000 00000 0.
FileInput.handl eChange(e)	000 00	0 00 000 00 onChange 0 0000 0.	0000 URL 000 00 0 000.
RatingInput	00 00	hover0 click 0000 0 0 000 00000.	000 rating 0 00 000.
handleDelete(id)	000 review id	mock 0000 00 00 0 00000.	00 000 000 00 0 00 00 0000.

00 00 00

- 0000 000, 000, 00, 000 00000.
- 00 00 0 FileInput 0 0000 00 00000 0000.
- 00 0 FormData 0 000 /createdReviews.json 00 0000.
- 00 0 0 state0 000000.
- 00 00: 00 00 000 0000 000 0000 mockReview.json 0 00000.
- 00 000 000 000 0000 000 000 0000.

00 000

- 00 00 00 → ReviewForm.values
- 000 00 → FileInput → values.imgUrl (File 00)
- 00 00 → RatingInput → values.rating
- 00 00 → createReviews() → /createdReviews.json
- 00 00 → mockReview.json → ReviewListItem

00 API

000	00	00 00	000 00 00 00 00	00 00
POST	https://mimo -49f6d- default- rtbd.firebaseio	FormData(con tent, imgUrl,	00	00 00 00

1111	11	11 11	1111 11 11 11 11	11 11
	eio.com/crea tedReviews.j son	nickname, rating)		
GET	https://mimo -49f6d- default- rtdb.firebas eio.com/revi ew.json	11	helper 11 body 11 11 111111 11 11 1111	1111 11 11 helper

API(Application Programming Interface, 111111 11.11 11)

11 111111 1111 1111 API 11111111. 11 1111 1111 11 11 11 111111.

11	1111	11	11 11	11 11(11 11)	11 11
1111	POST	accounts: signInWith Password	email, password, returnSec ureToken	idToken	component s/Auth/Aut hForm.jsx
111111	POST	accounts: signUp	email, password, returnSec ureToken	idToken	component s/Auth/Aut hForm.jsx
111111 11	POST	accounts: update	idToken, password, returnSec ureToken:f alse	11	component s/Profile/ ProfileFor m.jsx
11 11	GET	/itmes.js on	11	title, price, content, imgUrl	component s/Main/Mai nList.jsx

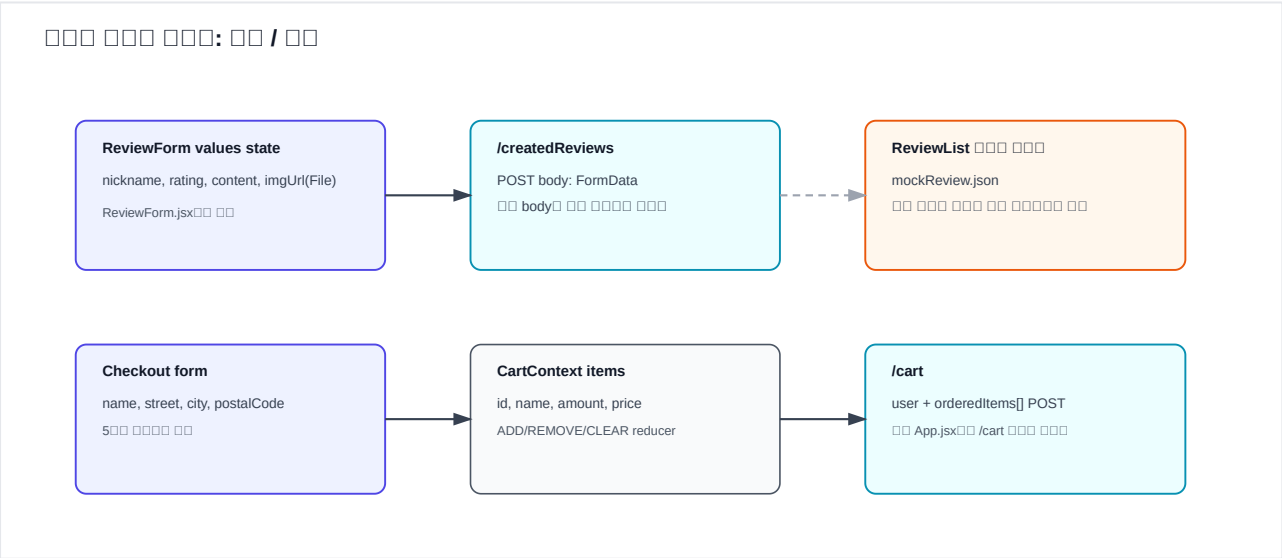
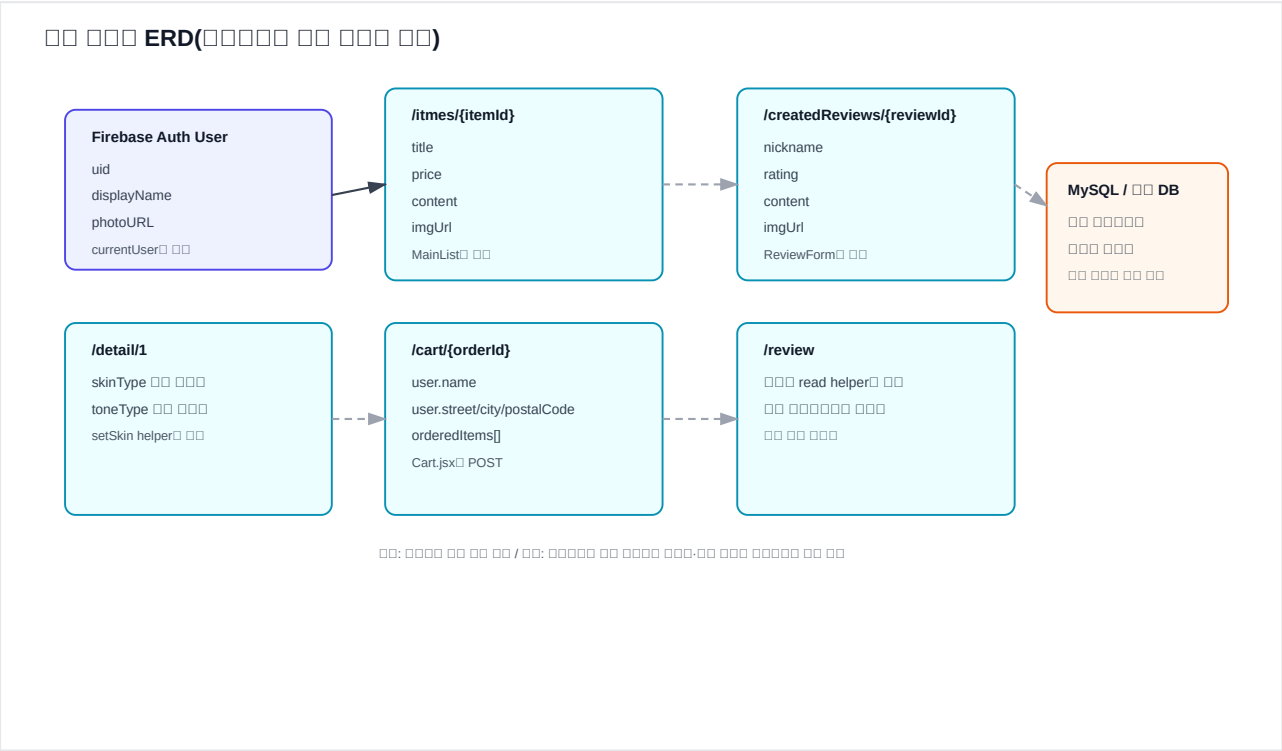
API	Method	URL	Request Body	Request Header	Response
POST	POST	/createdReviews.json	content, imgUrl, nickname, rating		components/Review/ReviewForm.jsx, data/api.jsx
GET	GET	/review.json		body	data/api.jsx
POST	POST	/cart.json	user, orderedItems		components/Cart/Cart.jsx
GET	GET	/detail/1.json		body	data/api.jsx
POST	POST	http://localhost:8000/api/v1/upload	FormData(file)	path	components/Modeling/camera.js, components/Modeling/upload.js
POST	POST	http://localhost:8000/api/v1/makeup	{ path }	prediction, imageArray	components/Modeling/camera.js, components/Modeling/upload.js

API 1000 1000

- localhost:8000 API 1000 1000 1000 1000 1000.
- API 1000 FormData 100000 Content-Type: application/json API 1000 1000 1000 1000.
- APIUtils.js 1000 1000(API_BASE_URL) 1000 1000 1000 1000 1000 1000.

DB(Database, 0000000) 0000 ERD(Entity Relationship Diagram, 000 000)

0 00000 SQL 000000, JPA 000, Prisma/TypeORM 00 00 00 000 000 0000. 000 00 00 00 000 00 00 **Firebase Realtime Database** 00 00 00 000000.



□□ □□□

□□	□□	□□(□□)	□□ □□	□□□/□□	□□ □□
/itmes/{itemId}	title	string	□□ □□□□ □ □□ □□	□□	/main
/itmes/{itemId}	price	string □□ number	□□□ □□	□□	/main
/itmes/{itemId}	content	string	□□□ □□	□□	/main
/itmes/{itemId}	imgUrl	string(URL)	□□□ □□	□□	/main
/createdReviews/{reviewId}	nickname	string	□□ □ □□□ □ □	□ □□□□ □ □ □□	/review
/createdReviews/{reviewId}	rating	number □□ string	□□ □ □□□ □ □	□□□ 0	/review
/createdReviews/{reviewId}	content	string	□□ □ □□□ □ □	□□□ □ □□□	/review
/createdReviews/{reviewId}	imgUrl	File/□□□ □□ □□□	□□	□□□ null	/review
/cart/{orderId}	user.name	string	□□ □□ □ □□ □□	□□ □□	□□□□ □□
/cart/{orderId}	user.street	string	□□ □□ □ □□ □□	□□ □□	□□□□ □□
/cart/{orderId}	user.city	string	□□ □□ □ □□ □□	□□ □□	□□□□ □□
/cart/{orderId}	user.postalCode	string	□□ □□ □ □□ □□	□□ 5	□□□□ □□

00	00	00(00)	00 00	000/00	00 00
/cart/{orderId}	orderedItems[]	array	00 0 00	000 000 context 00	0000 00
/detail/1	00 (skinType , toneType 00)	object	00	helper0 00	00 00 00 0 00
/review	00	object/array	00	helper0 00	00 00 00 0 00

00 00 00

- 00 00: 000 0000 00 00/00 00 000 000 00
- 00 00: 00 000 00 0000 0000 000 000 00000 0000 00 00
- 0000 00: README0 000 0000 0000 00/000 000 00 MySQL 00

00000 0 00 00

1) 00 00

00	00 00	00
00000 00	/home/runner/work/MIMO /MIMO/mimo-frontend	Create React App 00 00000 00.
000 00	mimo- frontend/package.json	react , react-router- dom , firebase , axios , react-webcam 00 0000 0.
00 00 00	mimo-frontend/.env	Firebase 00 00 00 000 00 000 00 CRA .env 000 0 00 000 0000 0000.

Item	File Path	Description
Firebase config	<code>mimo-frontend/src/fbase.js</code>	Firebase config file for the frontend.
Token storage	<code>localStorage['token']</code>	Token storage key in local storage.

2) Setup

Command	Script	Environment	Port	Notes
<code>mimo-frontend</code>	<code>npm install</code>	Node/npm	<code>node_modules</code>	Install dependencies
<code>mimo-frontend</code>	<code>npm start</code>	3000	<code>http://localhost:3000</code>	Start the application, ESLint/A11y errors will be shown
<code>mimo-frontend</code>	<code>npm run build</code>		<code>build/</code>	Build the application
<code>mimo-frontend</code>	<code>npm test</code>	Watch mode		Run tests in watch mode

3) File Structure

File Name	File Path	File Description
AuthForm	<code>components/Auth/AuthForm.jsx</code>	Auth form component, fbase.js, App.jsx
MainList	<code>components/Main/MainList.jsx</code>	Main list component, pages/Mainpages.jsx, Firebase /items.json
Camera	<code>components/Modeling/camera.js</code>	Modeling upload component, components/Modeling/constants.jsx, AI

Component Name	Component Path	Component Files
Profile/Use	<code>components/Profile/Use</code>	<code>components/Profile/ProfileContent.jsx</code> , <code>components/Profile/ProfileForm.jsx</code>
Review	<code>components/Review/ReviewForm.jsx</code>	<code>components/Review/FileInput.jsx</code> , <code>components/Review/ReviewList.jsx</code> , <code>data/api.jsx</code>
Footer Nav	<code>components/Layout/nav/FooterNav.jsx</code>	<code>App.jsx</code> Footer Nav

4) Component Structure

- The `App.jsx` component is the main entry point of the application.
- Firebase DB is connected to `MainList`, `ReviewForm`, `Cart` components.
- `/cart` component is connected to `App.jsx`, `MainNavigation.jsx`, `FooterNav.jsx`, `Cart` component.
- The `camera.js` and `upload.js` components are used to capture and upload images using OpenCV.

5) Build & Run

- The build process is: `npm install`, `npm run build`, `npm test`.
- The run process is: `npm start` or `npm run start`.
- The test process is: `npm test` or `npm run test`.
- The deployment process is: `npm run deploy` or `npm run build` followed by `npm run deploy`.

6) Deployment

The deployment process involves creating a `Dockerfile`, setting up a CI/CD workflow, and deploying the application to a cloud provider.

- The deployment process is: `npm install` followed by `npm start`.
- The database is connected to Firebase.
- The application is deployed to `localhost:8000` or a cloud provider.

7) ○○○ ○○ ○○ ○○○

○○	○○ ○ ○○	○○ ○○
○○○ ○ ○○ ○○○ ○ ○○	<code>store/auth-context.jsx</code> , <code>App.jsx</code>	<code>localStorage['token']</code> ○○ ○○, <code>isLoggedIn</code> ○○ ○ ○
○○○ ○○○ ○○○ ○○○ ○○	<code>AuthForm.jsx</code>	Google ○○○ ○○ ○ <code>navigate()</code> ○ ○○○○ ○○ ○ ○ ○○ ○○
○○○○ ○○○○ ○○○ ○○○ ○ ○ ○	<code>App.jsx</code>	<code>/cart</code> ○○○ ○○○ ○○○○○○ <code>/main</code> ○○ ○○○○ ○○
○○ ○○○ ○○ ○○	<code>MainList.jsx</code>	<code>/itmes.json</code> ○○ ○○○ <code>response.ok</code> ○○
○○○ ○○○○○ ○○○ ○ ○○	<code>Review.jsx</code> , <code>ReviewForm.jsx</code>	○○○ <code>mockReview.json</code> ○ ○○○ ○○ ○○○ ○○○○ ○○ ○○ ○○
○○○ ○○ ○ ○○○ ○ ○	<code>camera.js</code> , <code>upload.js</code>	<code>localhost:8000</code> ○○ ○○, <code>path</code> ○○, <code>prediction/imageArray</code> ○○, <code>this.setState</code> ○○ ○ ○
○○○ ○○/○○○ ○○ ○○	<code>UserProfile.jsx</code> , Firebase Auth	<code>currentUser</code> ○○○ ○○○ ○ ○○○ ○○○ ○○○○ ○○

○○ ○○

- Markdown ○○○○ ○○○○ ○○ ○○○ ○○○ `docs/handover/images/` ○○ ○○○ ○○○○○.
- ○○ ○○○ ○○○ anchor id(`#architecture` , `#screen-auth` ○)○ ○○○○○○.
- PDF○ ○○ Markdown○ HTML○ ○○○○ ○ ○○○○ ○○○○ ○○ ○○○○, ○○○ ○○○ ○○○ ○○ ○○○
○○ ○○○○○.